Altu Bozyaka (s2572629)

Computer Graphics: Rendering - Coursework Checkpoint 2

As part of Module 2,

- **I built:** four new pieces on top of my Module 1 renderer: an orthographic camera, supersampling , three ways of placing shadow rays on an area light (uniform, grid and Halton), and a branching factor (BF) for the distributed ray tracer. They are all set from the scene file, like everything else.
- **How I show it:** each task has its own scenes that I designed to make the effect obvious. The yellow numbered markers like **1** point at the exact spot I am talking about, and I measured everything with numbers (sizes in pixels, errors against a reference image, render times) instead of only judging by eye.
- **Checking:** one script (module2_study.py) renders every image and writes all the numbers in this report to output/module2/. Another (validate_module2.py) checks the new features against exact answers or pbrt-v4, and all 10 of its checks pass.

2 Camera models

2.1 Orthographic vs perspective projection

What I did and why: The two cameras differ in one thing: where the rays start. A perspective camera fires every ray from a single point (the eye), so the rays fan out. An orthographic camera fires parallel rays from a flat rectangle. To explain this mathematically, if a point is at (x, y, z) in front of the camera, it lands on the image at

perspective: $u = \frac{W}{2} + f\frac{x}{z}$ orthographic: $u = W\frac{x - w_0}{w_1 - w_0}$

where f = (H/2) / tan(fov/2) is the focal length in pixels and [w_0, w_1] is the width of the orthographic window in scene units (the formula for the vertical position v is the same with y). The whole difference is the division by z. In a perspective camera, an object twice as far away looks half as big, and parallel lines going into the distance meet at a vanishing point. In orthographic projection the depth z does not appear at all, so things keep the same size however far away they are, and parallel lines stay parallel. In my code, the perspective ray starts at the eye, while the orthographic ray starts at the matching point on the window and points straight ahead. I copied pbrt-v4's conventions exactly, so the same scene file gives the same picture in both renderers.

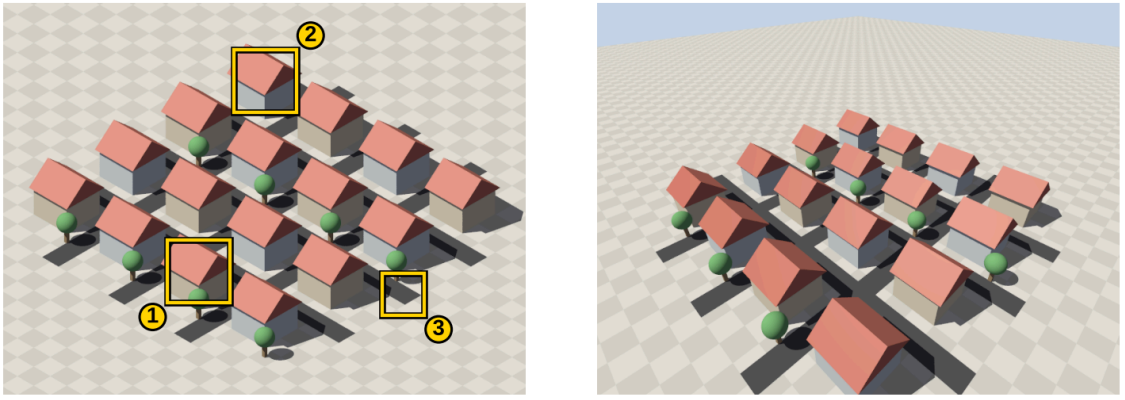
Scene where orthographic is better (camera_ortho): a town block of 16 identical houses (Fig. 1a). This is the kind of view a map, a city plan or a strategy game uses, where you want to compare buildings and understand the layout, not feel like you are standing in the street and looking ahead. My script found the red roofs in the image and measured them: in the orthographic view all 16 roofs are 4686 ± 21 pixels, so the near house **1** and the far house **2** really are drawn the same size, and the streets **3** stay parallel. With a perspective camera from the same direction (Fig. 1b), the roofs range from 1447 to 13156 pixels, so the closest house looks 9 times bigger than the furthest one and the neat grid turns into a trapezoid.

Scene where perspective is better (camera_persp): a railway running to the horizon, with identical telegraph poles every 10 units (Fig. 1c). Here depth is the whole point I was trying to show: the rails meet at a vanishing point **1**, and the poles **2** shrink with distance (607, 235, 145 and 83 pixels tall at 0, 10, 20 and 40 units away) compared with the nearest pole **3**. When I rendered the same camera as orthographic (Fig. 1d), every pole came out exactly 416 pixels tall, so you can no longer tell which one is closer, the rails stop converging and the horizon disappears completely.

How I tested it: I rendered a test scene with both my renderer and pbrt-v4's orthographic camera. Only 30 of 43 200 pixels (0.069 %) landed on a different side of an object's edge, and the shading differed by 0.3 %. A second check puts two equal spheres at 2 and 12 units away: orthographic draws both 80 pixels wide exactly as predicted, while perspective draws them 100 and 20 pixels wide.

(a) camera_ortho: orthographic, isometric direction
roof area 4686 ± 21 px over all 16 houses

(b) perspective (fov 62°) from the same direction
roof area 4202 ± 2892 px (min 1447, max 13156)



(c) camera_persp: perspective, fov 45°
pole heights 607 / 235 / 145 / 83 px at z = 0 / 10 / 20 / 40

(d) orthographic, same position and direction
every pole 416 px tall: depth cannot be read

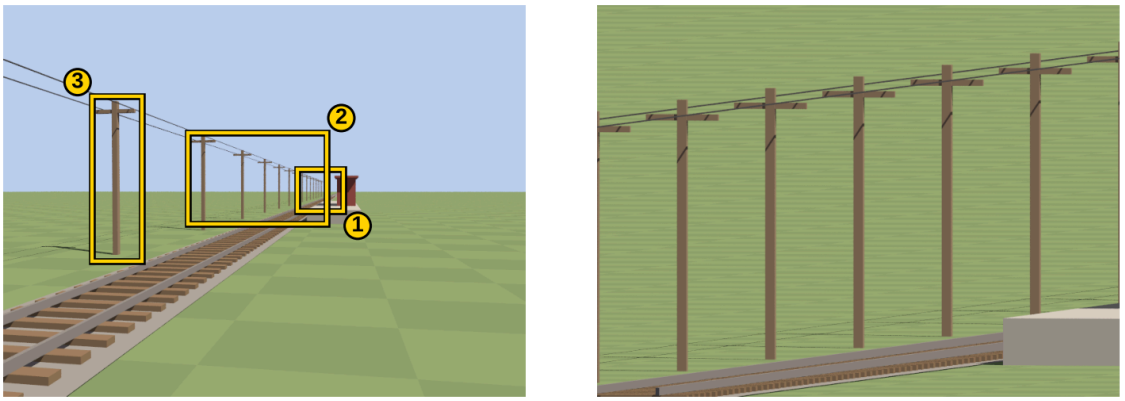


Figure 1: Orthographic vs perspective. In the town, orthographic keeps all 16 identical roofs the same size (**1** near, **2** far) and the streets parallel (**3**), while perspective from the same angle makes the near roofs up to 9× bigger. On the railway, perspective shows depth through the vanishing point **1** and the shrinking poles **2** (compare with **3**). Orthographic makes every pole 416 px tall, so all sense of depth is lost.

2.2 Field of view

What I did and why: For this I built a gallery of identical spheres on a checkerboard floor in front of a tiled wall. The red sphere is my reference, 10.1 units from the camera, and two extra spheres sit close to the camera on each side. I kept the camera in exactly the same place and only changed the field of view: 25°, 60° and 100° (camera_fov1/2/3).

How I tested it: From the formula above, an object's size in the image is proportional to f, which is proportional to 1/tan(fov/2). So going from 25° to 100° should make the red sphere 5.4 times smaller. My script measured its width as 134, 51 and 24 pixels, against predictions of 134.8, 51.8 and 25.1 (Fig. 2abc). A separate validation check on a flat disc at 20°, 40° and 80° matched my prediction within half a pixel. Inreality, a narrow field of view works like a zoom lens, the subject fills the frame and the floor tiles look big. A wide one fits in much more of the room, but everything in the middle becomes small.

Observation (distortion): At 100° the two near spheres **2** **3** are no longer round. They are stretched outwards into ovals 1.42 times longer than they are wide, while the red sphere **1** in the middle stays perfectly round. They sit 45.5° away from the centre of the view, and a flat image stretches things there by about 1/cos(45.5°) = 1.43, which matches what I measured. A pinhole camera keeps straight lines straight, and the price is stretching near the edges, which gets worse very quickly as the field of view approaches 180°. At 25° and 60° nothing is far enough from the centre to show it.

I also noticed a second effect that people often blame on the field of view. In the bottom row (Fig. 2def) I moved the camera so that the red sphere stays the same size ("dolly zoom"). At 25° the camera is 26 units away and the rows of spheres look squashed together, while at 100° it is only 4.9 units away and they look spread far apart. So the "flattened" or "exaggerated" look really comes from how far away the camera is. The field of view only decides how far you have to stand to frame your subject.

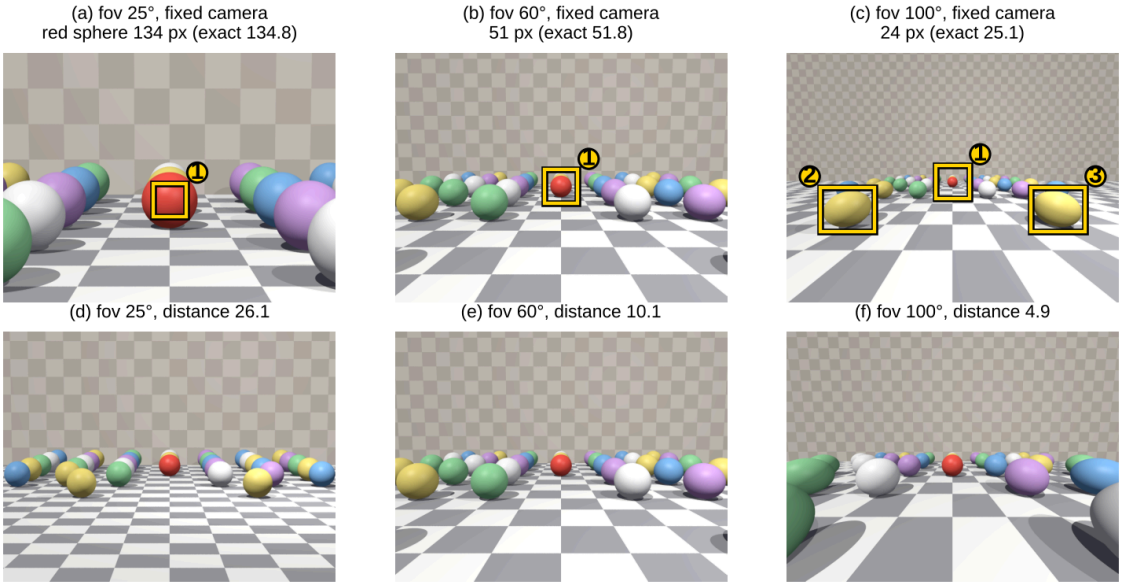


Figure 2: Field of view. (abc) Same camera position: the red sphere **1** shrinks from 134 to 24 px as the field of view widens, and at 100° the near spheres **2** **3** are stretched into ovals. (def) Dolly zoom: the camera moves so the red sphere stays about 52 px wide, and the spacing of the rows shows how distance (not the field of view itself) causes the flattened or stretched look.

3 Antialiasing

3.1 Demonstrating aliasing

What I did and why: Aliasing happens when the scene has detail finer than a pixel, so I decided to pack several kinds of fine detail into one garden scene (Fig. 3): a checkerboard lawn running to the horizon **1**, three power lines much thinner than a pixel **2**, a picket fence with thin slats **3**, a wheel with 36 spokes **4**, a checkered ball **5** and a textured crate **6**. With one sample per pixel, both tracers fire exactly one ray through the centre of each pixel, so they only look at the scene through a regular grid of points.

What goes wrong: In aa_wsrt the lawn breaks into wavy bands **1**, because the regular pixel grid clashes with the regular checker pattern. The power lines **2** turn into dashes because a line thinner than a pixel is either hit by the centre ray or missed completely. The fence slats **3** merge into a messy pattern, and the spokes **4**, the ball's checks **5** and the crate edge **6** all have jagged staircase edges.

Observation: aa_drt uses the DRT with one camera ray per pixel but 32 bounce rays at every hit, and it has exactly the same artefacts (plus some noise). At first I expected the extra rays to smooth things out, but they can't. All of them start from the single point where the camera ray hit, so they average the lighting at that point but still only see one point of the geometry per pixel. Only spreading samples across the area of the pixel fixes aliasing.

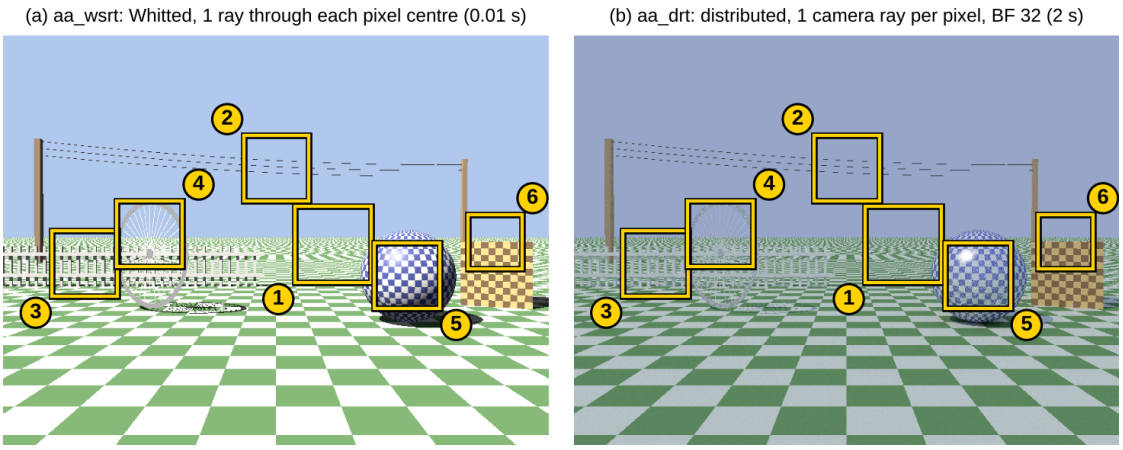


Figure 3: Aliasing with one ray per pixel: **1** wavy pattern on the lawn, **2** power lines broken into dashes, **3** fence slats merging, **4** broken spokes, **5** distorted checks, **6** stair-stepped crate edge. The DRT's 32 bounce rays per hit (b) don't remove any of them.

3.2 Supersampling

What I did and why: Supersampling means firing several rays through different points of each pixel and averaging their colours. I added pbrt's three samplers: independent (random points), stratified (the pixel is cut

into an $n \times n$ grid with one point per cell, either jittered randomly inside its cell or exactly at the centre) and halton (a low-discrepancy sequence whose points spread out evenly by design). For the required images aa_ss1/4/9/16 I used the jittered stratified sampler, because it gets the best of both: the grid keeps the samples evenly spread, and the small random jitter turns leftover aliasing patterns into fine noise, which looks much less distracting.

How I tested it: I compared each image with a very clean reference rendered at 1024 samples per pixel (spp) and measured the error as RMSE, roughly the average colour difference per pixel. Fig. 4 shows close-ups of all six regions and Table 1 gives the numbers. Four samples already join the power lines 2 back into faint continuous lines and turn the lawn's wavy patterns (moiré pattern) 1 into fine noise, and by 16 samples the spokes 4 and crate edge 6 look smooth. Over the whole image the error drops from 0.0735 to 0.0133, 5.5 times lower. The far lawn 1 and the fence 3 stay the worst regions even at 16 spp, because their detail is so fine that only the 1024-sample reference blends it into a flat green-white colour.

Observation: I also tried the other two patterns at the same sample counts (bottom of Table 1). At 16 spp jittered wins clearly (0.0133), ahead of the regular grid (0.0198) and random points (0.0193). The regular grid is the worst of all on the lawn 1, because it only moves the moiré to a finer scale instead of removing it, while random points leave gaps that show up on the thin lines 2. What surprised me is that on a single straight edge, in a separate validation test, the regular grid actually converges fastest of all. So whether a regular grid is good depends on whether the scene has repeating patterns that it can clash with.

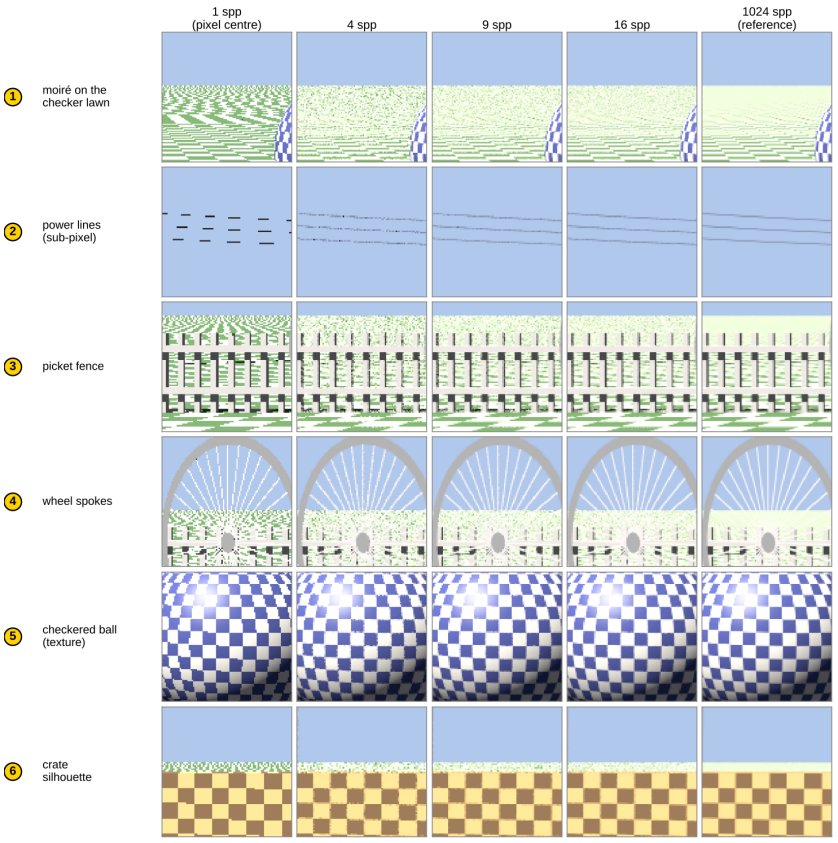


Figure 4: Supersampling close-ups (WSRT, jittered), from aa_ss1, aa_ss4, aa_ss9, aa_ss16 and a 1024-spp reference. The power lines 2 reconnect from 4 spp, the edges 4 6 are smooth by 16 spp, and the lawn 1 is still noisy at 16 spp.

Table 1: Error (sRGB RMSE, lower is better) against the 1024-spp reference, for the whole image and each region of Fig. 3.

pattern	spp	whole	1 lawn	2 lines	3 fence	4 spokes	5 ball	6 crate
pixel centre	1	0.0735	0.169	0.061	0.167	0.127	0.076	0.082
jittered	4	0.0336	0.073	0.019	0.082	0.056	0.035	0.039
jittered	9	0.0194	0.044	0.008	0.049	0.035	0.019	0.026
jittered	16	0.0133	0.031	0.006	0.034	0.024	0.013	0.017
regular grid	4	0.0373	0.091	0.014	0.098	0.066	0.031	0.044
random	4	0.0405	0.085	0.026	0.094	0.065	0.048	0.045

pattern	spp	whole	1 lawn	2 lines	3 fence	4 spokes	5 ball	6 crate
regular grid	16	0.0198	0.051	0.007	0.052	0.039	0.014	0.027
random	16	0.0193	0.040	0.010	0.046	0.031	0.023	0.021

3.3 Performance and cheaper alternatives

How I measured: I timed every render 5 times and took the median (Fig. 5). The WSRT's render time grows in a straight line with the number of samples. It takes about 0.0128 s per sample per pixel: 0.011 s at 1 spp, 0.205 s at 16 and 0.821 s at 64. That makes sense, because every extra sample traces a whole new ray tree, and the ray count went up by exactly the same factor (0.72 M rays at 1 spp, 11.6 M at 16). The DRT version also grows in a straight line, but each sample costs about 216 times more (2.76 s), because of all the bounce rays.

The trade-off: The cost goes up much faster than the quality. Across the whole image the error falls roughly like $\text{spp}^{-0.6}$, so halving the error costs about 3 times the render time. By 16 spp the image looks clean apart from the far lawn, but fixing that would take hundreds of samples. On top of that, most samples are wasted: in the sky and the near lawn the very first sample is already right.

Cheaper alternatives (I did not implement these, potential exceptionalism in the future):

- **Adaptive sampling:** start every pixel with a few samples, and only add more where the samples disagree a lot. Here that would put the effort into the lawn, fence, wires and edges and leave the sky at 4 spp. The risk is that a wire missed by all the first samples never gets refined.
- **Texture filtering (mip-mapping):** most of the leftover error is the lawn texture. If the texture lookup returned the average colour over the area a pixel covers, the moiré texture would disappear at 1 spp for the price of a single lookup.

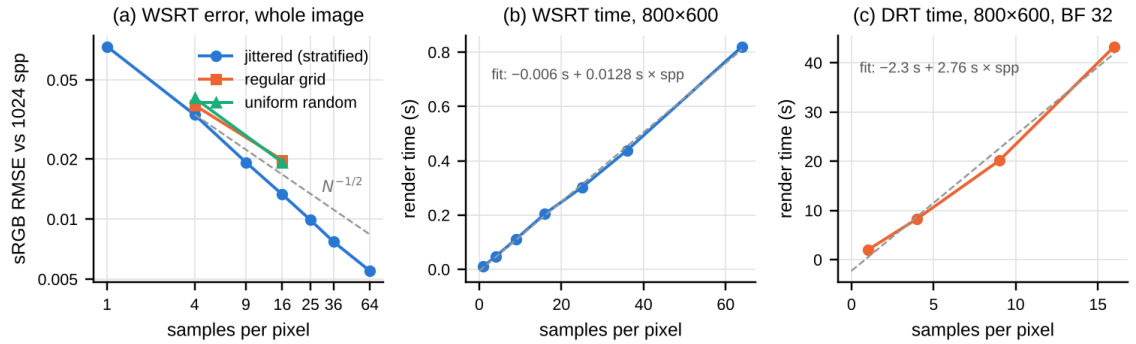


Figure 5: (a) Error against samples per pixel (log scale): jittered is the best pattern at both 4 and 16 spp. (b, c) Render time grows in a straight line with spp for both tracers (I took the median of 5 runs).

4 Sampler comparison

4.1 How I set it up

What I did and why: In the DRT, every shading point sends N shadow rays to the area light, and the new setting "string lightsampling" chooses where on the light they go:

- **uniform:** N completely random points
- **grid:** the light is cut into an n × n grid and one ray goes to the centre of each cell (so it is the same every time, with no randomness). If N is not a square number it is rounded, so 10 becomes 3×3, with a note in the log.
- **Halton:** the first N points of the 2D Halton sequence, which fill the square evenly without lining up in rows.
- **stratified:** one random point per grid cell, which I added as an extra comparison.

Two details needed some attention from me. First, my light is a rectangle made of two triangles. Picking a triangle first and then a point inside it would split the grid in half, so the renderer recognises the rectangle and places the samples on the rectangle itself, which makes the grid cells real cells of the light. Secondly, if every pixel used exactly the same Halton points, neighbouring pixels would make the same mistake and you would see patterns. So each shading point shifts the whole set of points by a random offset (a "Cranley-Patterson rotation"), which keeps them evenly spread but makes the errors random.

The test scene: A rectangular light hangs over a grey floor with five objects, each chosen to show the patterns in a different way (Fig. 6a): a box lined up with the light's edges **1**, a box rotated by 30° **2**, a sphere with a curved shadow edge **3**, a thin pole with a narrow shadow **4**, and a grille of seven thin slats whose soft shadows overlap **5**. I rendered with one camera ray per pixel and direct light only, so the shadow rays are the only source of randomness and any error comes from the sampling alone.

How I measured it: I compared only the pixels in the soft shadow edges since that is the only place the pattern matters. Each pattern was rendered at N = 1, 4, 16, 64, 256 with 24 different random seeds, and for each case I computed the RMSE against a reference. I rendered the reference in pbrt-v4 with 16 384 samples per pixel (636 s), and also with my own renderer for comparison. The two references agree almost perfectly: their difference is about a third of the smallest error I measured, so the reference isn't affecting the comparison.

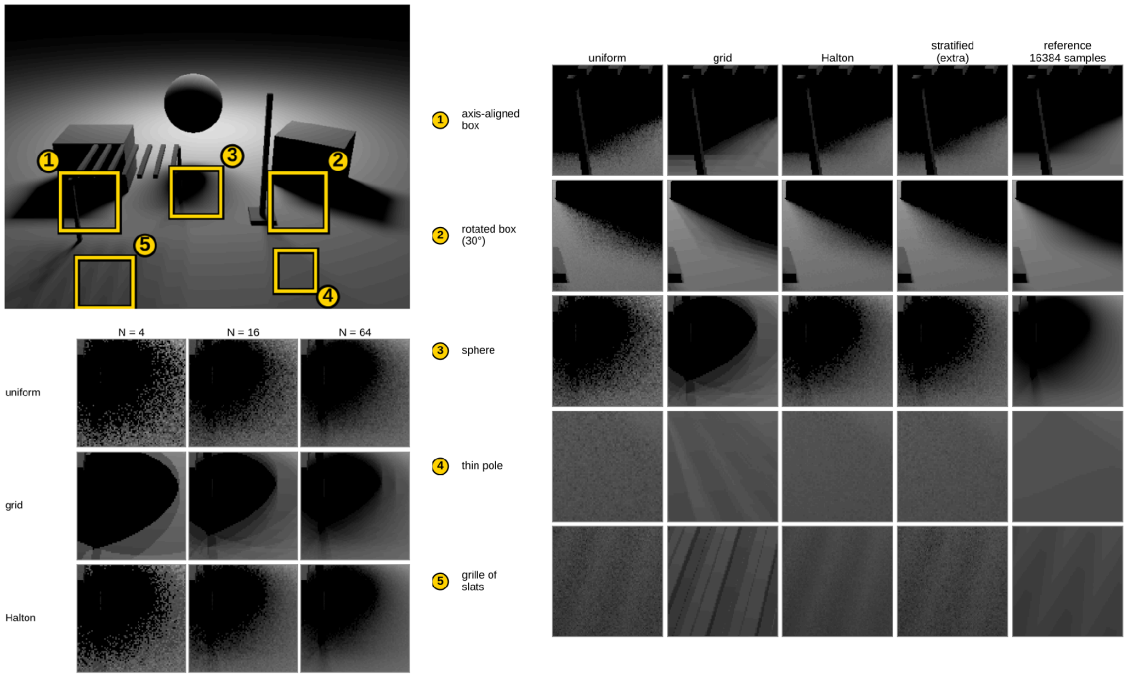


Figure 6: Sampler comparison (sampler_unif/grid/halton_4/16/64). With the same number of rays, uniform gives some grain. The grid has no grain at all but splits each soft shadow into visible bands, clearest on the sphere **3** and as streaks on the grille **5**. Halton gives the finest grain. In (c) the grid's bands are still visible at N = 64, while Halton is almost smooth.

4.2 Results

Table 2: Error (RMSE) in the soft-shadow pixels against the pbrt-v4 reference, averaged over 24 seeds (lower is better). For the grid, all of the error is a fixed offset (bias) rather than random noise.

pattern	N=1	N=4	N=16	N=64	N=256
uniform	0.0702	0.0351	0.0176	0.0088	0.0044
grid	**0.0504**	0.0239	0.0090	0.0046	0.0021
Halton	0.0701	**0.0230**	**0.0076**	**0.0027**	**0.0010**
stratified (extra)	0.0701	0.0256	0.0097	0.0035	0.0013

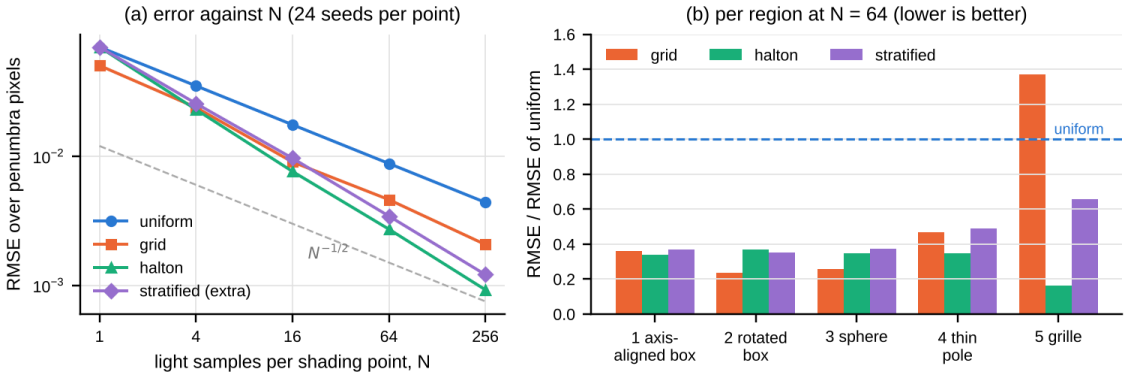


Figure 7: (a) Error against the number of shadow rays N (log scales). Uniform halves its error every time N goes up 4×, Halton and stratified improve faster, and the grid levels off. (b) Error in each region at N=64 compared with uniform: the grid is the best on the boxes and sphere **1**–**3** but worse than uniform on the grille **5**.

Uniform: it behaves exactly like the Monte Carlo theory from the lectures says: the error halves each time N goes up 4 times (0.0176 → 0.0088 → 0.0044), and the error is pure random noise. It always looks the grainiest (Fig. 6b, first column).

Grid: it has no noise at all, which is great, but its error never averages away, because every pixel looks at the light through the same few points. As a shadow edge moves across the light, the amount of light jumps in steps, and that produces the visible bands in Fig. 6: 16 bands per shadow edge at N = 16, still visible at N = 64 **3**. On straight, wide shadow edges these steps are small, so at N = 64 the grid actually beats every other pattern on the two boxes **1** **2** and the sphere **3** (Fig. 7b). The grille **5** is where it falls apart: the evenly spaced slats line up with the evenly spaced samples, so the error gets stuck. There, it is 1.37 times worse than uniform and 8.7 times worse than Halton.

Halton: it had the lowest error from N = 4 upwards, and its error falls faster than uniform's, so the gap grows with N. At N = 256 it is 4.5 times lower than uniform. Like the grid, its points are spread evenly, which keeps the noise low, but the random offset means it never makes bands and stays correct on average. It also handled the grille well, where the grid failed. At N = 1 it is the same as uniform (one random point), which the table confirms.

4.3 When each pattern is the better choice

- **Uniform** is the safe default choice in my opinion. Always correct on average, works for any N, and never makes patterns. But it needs 4 times more rays every time you want to halve the noise.
- **Grid** suits a small, fixed number of rays when noise is not acceptable but a little banding is, for example a quick preview, or a scene with big soft shadows from straight objects lined up with the light. I would avoid it with repeating objects like blinds, grilles or leaves, and whenever the image has to be super accurate, because adding rays only shrinks the bands but never removes the bias.
- **Halton** was the best choice for every object in my scene once N ≥ 4, and its benefit grows as N goes up. The only time I wouldn't bother is N = 1, where it is no better than uniform.

5 DRT parameter study

5.1 What BF and MD do, and what they cost

What I did and why: At every hit on a diffuse or glossy surface, my DRT now fires BF bounce rays in random directions and averages what they bring back. So one camera ray becomes up to BF rays at depth 1, BF^2 at depth 2 etc., until the maximum depth MD. Mirrors and glass are the exception as they reflect in exactly one direction so extra rays would all follow the same path and a mirror always casts one ray. I checked that the counts come out exactly as expected (for BF 3: 1, 3, 9, 27, 81 rays per level).

The two settings fix different problems:

- **MD** decides how many bounces light is allowed to take. If it is too low, light that needs more bounces is simply missing and the image comes out too dark. More samples won't fix low MD problem.
- **BF** decides how many rays estimate each bounce. A higher BF makes the image less noisy (noise falls like $1/\sqrt{BF}$, which my validation confirmed. BF 4 halved the noise and BF 16 quartered it) but doesn't change the brightness.

The cost: With BF 1, a deeper MD is cheap, because each level adds just one ray. With BF above 1 the cost explodes exponentially, and most of it goes on the deepest levels, which add the least light.

How I picked the values: For each scene I rendered a very clean reference, then swept MD with BF fixed, and BF with MD fixed. For every render I measured how much of the reference's light it captured (to spot images that are too dark), how noisy it was (by rendering it twice with different seeds and comparing), and how long it took. Then I picked the smallest MD that captures nearly all the light (or the depth a visible feature needs), and a BF where the drop in noise was worth the extra time. I designed the four scenes so that each one naturally needs one of the four combinations (Fig. 8).

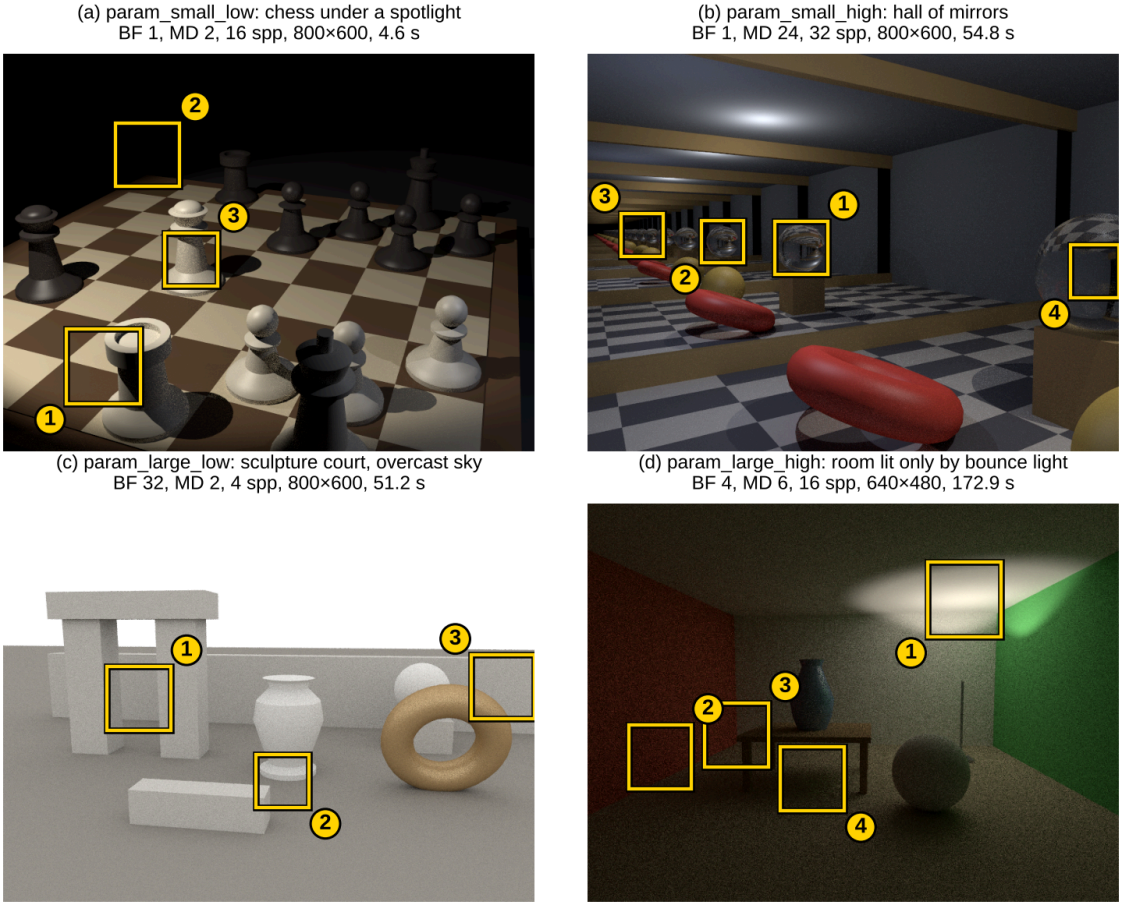


Figure 8: The four parameter scenes. (a) Chess scene (b) Mirrors scene (c) Sculptures scene (d) Room scene

5.2 The four scenes

Small BF, low MD (param_small_low: BF 1, MD 2, 16 spp, 4.6 s): a chess endgame under a spotlight in a dark, open room. Nearly all the light is direct **1**, the dark surroundings **2** bounce almost nothing back, and the only bounce light worth having goes from the board onto the bottom of the pieces **3**. The sweep agrees: MD 1 already captures 93.8 % of the light, MD 2 98.2 % and MD 3 99.6 %, and the error was lowest at MD 2. A bigger BF hardly helped: BF 16 only cut the noise from 0.088 to 0.059 and took 6.6 times longer, because most of the noise here comes from the direct light, which BF doesn't touch.

Small BF, high MD (param_small_high: BF 1, MD 24, 32 spp, 55 s): a hall of mirrors: two parallel mirrors with a glass sphere, a red torus and two lights between them. Every extra reflection needs one more level: the glass sphere's first reflection **1** needs MD 2, the second **2** needs MD 3, and the long corridor **3** needs many more (Fig. 9, top right). The light captured goes 32 %, 72 %, 86 % and 98 % at MD 1, 2, 3 and 6, and 99.9 % at MD 12. A big BF is wasted here because mirror bounces never split and BF 2 took 8 times longer and only reduced the noise from 0.171 to 0.124. With BF 1, on the other hand, a deep MD costs very little (15.4 M camera rays, with 7.5 M still bouncing at depth 23). Measured purely by error per second the best value was MD 8, but I chose 24 so the far end of the corridor **3** is actually drawn instead of cut off. That was a just a visual choice, and it cost me 3x the render time.

Large BF, low MD (param_large_low: BF 32, MD 2, 4 spp, 51 s): an open sculpture court under a flat sky, with no other lights. In my renderer the sky is not sampled as a light. It is only found by bounce rays that escape into it. So BF is the only thing that reduces the noise on the big softly lit surfaces **3** and in the soft shadows **12**. Since the space is open, extra bounces add very little (at BF 4: 94.4 %, 99.0 % and 99.8 % of the light at MD 1, 2 and 3), so MD 2 is enough. Raising BF from 1 to 4, 16 and 64 cut the noise from 0.218 to 0.107, 0.056 and 0.034: close to the expected $1/\sqrt{BF}$.

Large BF, high MD (param_large_high: BF 4, MD 6, 16 spp, 173 s): a closed room whose only light is a floor lamp pointing at the ceiling **1**. Everything else is lit by light bouncing between the walls, including colour bleeding from the red and green walls **2**, so the scene needs both settings. The light captured rises 37 %, 66 %, 81 %, 90 %, 97 % and 99 % for MD 1, 2, 3, 4, 6 and 8, and the corner **3** and the area under the table **4** stay dark until several bounces are allowed. Raising BF at MD 6 reduces the noise from 0.96 (BF 1) to 0.59, 0.45, 0.39 and 0.31 (BF 2, 3, 4, 6), but the number of rays grows from 2 million (0.03 s) to 3.1 billion (93 s) (Fig. 9, bottom). The final image uses BF 4 as it already needs 7.3 billion rays and 173 s, and BF 6 would involve 7x more rays. Even then, it is still visibly noisy.

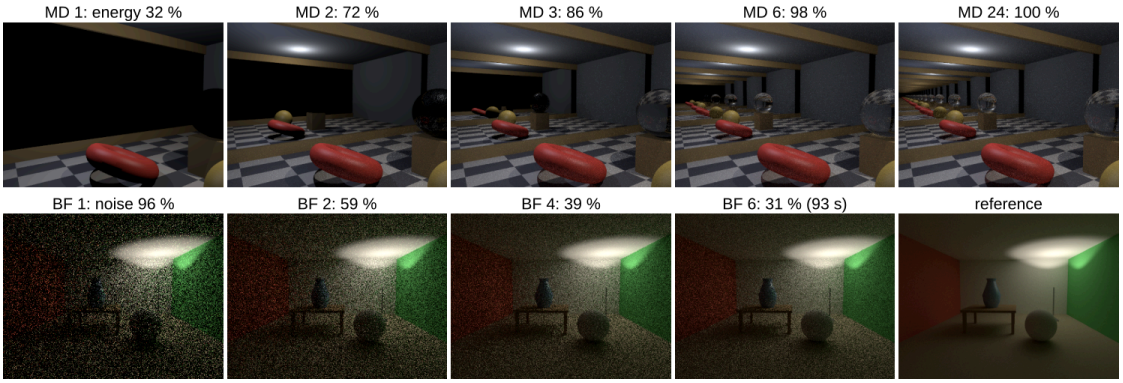


Figure 9: Top: mirrors at BF 1 with increasing MD. Each extra level adds another reflection to the corridor. Bottom: room at MD 6 with increasing BF.

5.3 Is a big BF actually worth it?

The MD results were what I expected basically match MD to how far light has to travel in the scene. That means 2 for open, directly lit scenes, 6-8 for closed rooms, and as many levels as reflections you want to see. With BF 1 this is cheap. The BF results surprised me, though. For the two big-BF scenes my script also did an equal-time test: render with BF 1 but as many samples per pixel as fit in the same time. In the sculpture court, BF 32 × 4 spp (14.4 s) had an error of 0.0465, while BF 1 × 882 spp (14.6 s) had 0.0235, half as much. In the room, BF 4 × 4 spp (7.7 s) had 0.389, against 0.084 for BF 1 × 772 spp (6.3 s), 4.6 times better. In all four BF sweeps, the error per second of render time was best at BF 1.

I think there are three reasons.First, extra samples per pixel improve everything at once (edges, the lens, the shadow rays and every bounce), while BF only adds rays to the bounce light. Secondly, the BF tree spends most of its rays at the deepest levels, which carry the least light. Thirdly, the BF rays from one hit all share the same camera ray and the same first hit point, so they aren't as independent as separate samples. So in my renderer, a big BF suits the sculpture and the room only in the sense that these are the scenes where it changes the picture most at a fixed number of samples per pixel. It isn't the cheapest way to get a clean image. I kept the required settings, but honestly, BF 1 with more samples would have been better in both.

6 Problems I ran into

Table 3: Issues I had to handle in Module 2, and how each was solved.

Issue	Why it was a problem	What I did
grid shadow rays on a rectangle light	the light is two triangles, so picking a triangle first splits the grid in half	detect the rectangle and place samples on it directly, so grid cells are real cells
grid needs a square number	N = 10 can't form an n × n grid	round to the nearest square (3×3) and write it in the log
wasted rays at the last level	with no sky in the scene, rays at the deepest level can never bring back any light	skip them: identical images, fewer rays
equal-time test kept missing	the first guess of the matching spp came from a very short test render	re-measure once and rescale until the two times agree within 10 %
new random sampling patterns	different threads must still give the same image	added a check that 1 thread and all threads give identical images

7 Use of generative AI

Tools and how much it did: As in Module 1, most of the Module 2 code was produced by Claude Code (Claude Opus 5.5) in VS Code: the orthographic camera, the samplers, the light sampling patterns, the branching factor, the scene generator and the study and validation scripts. I wrote very little code by hand. My part was deciding what each task needed, giving the formulas from the lectures (the projection equations, the Halton sequence, $1/\sqrt{N}$ noise), designing the scenes so each effect would be obvious, and re-prompting when a scene didn't show its feature clearly. Claude also helped me understand the results, for example why the grid fails on the grille and why a large BF lost the equal-time test.

8 Summary

Everything Module 2 asks for works and is measured: An orthographic camera that matches pbrt-v4, supersampling with three samplers, three shadow-ray patterns (plus a fourth extra one), and a branching factor that behaves exactly as expected. The results I find most interesting are that a regular grid is the best pattern on a single edge but the worst on repeating patterns, both for antialiasing and for soft shadows, that Halton was the best way to place shadow rays, and that a big BF lost to more samples per pixel even in the scenes designed for it. The main limitations are still that there is no BVH yet, so scenes with meshes are slow, and that bounce rays are sampled uniformly. The BVH comes in Module 3 and importance sampling in the final phase.